

2-DAY HANDS-ON WORKSHOP · KRISA 2.0-ALIGNED

Mastering Software Quality Assurance

Modules 1 & 2

The Shift-Left phase, building quality before a line of code is written.
One continuous project: ShopFast checkout.

MODULE 1 · FOUNDATION

**The Quality Mindset**

MODULE 2 · PLAN

**QA Planning & Proactive Quality**

Jadual slot: Modul 1: Minda Kualiti

Run sheet / session timing

Hari 1, 09:30 hingga 11:00 · 90 minit · kira-kira 80% amali



Masa	Segmen	Tempoh
09:30 hingga 10:05	Konsep	35 minit
10:05 hingga 10:35	Aktiviti amali	30 minit
10:35 hingga 10:55	Bengkel AI W01	20 minit
10:55 hingga 11:00	Rumusan	5 minit

Tiga panduan JDN yang saling melengkapi

Three complementary JDN guides

KRISAv2 Bab 8 8.2: KRISA, PPrISA dan Software Quality Management Using DevOps Approach

KRISAv2

Bagaimana membina sistem

6 fasa, aktiviti F1.1 hingga F6.4, templat D01 hingga D18, pengujian Unit hingga FAT

PPrISA 2.0

Bagaimana mengurus projek

Tadbir urus, pelan kualiti, risiko, isu, pindaan, laporan status, penamatan (PPrISA01 hingga 18)

DevOps SQM

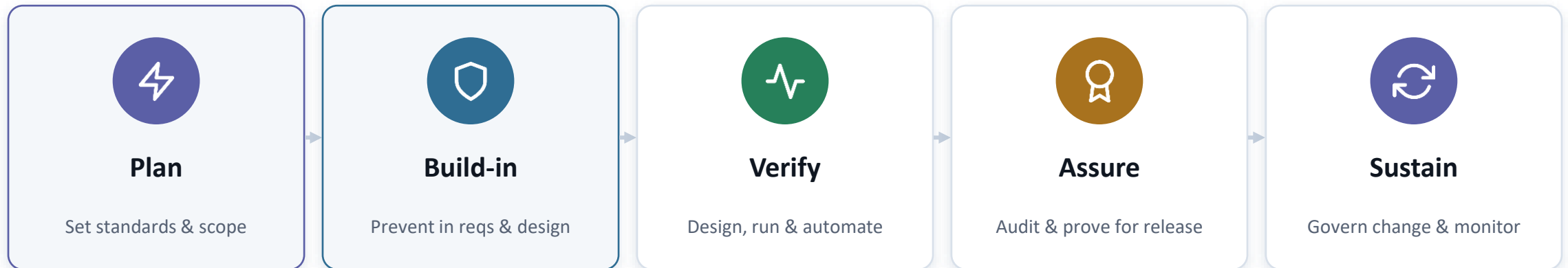
Bagaimana menyampaikan dengan pantas dan berkualiti

CI/CD, automasi ujian, Gate Review, DoD, APM dan pemantauan

SQA menyambung ketiga-tiganya: pencegahan (semakan, pelan kualiti) dan pengesanan (ujian, metrik) di setiap Gate Review.

Quality flows through five phases

Modules 1 & 2 establish the front of that flow, the Shift-Left foundation. You set the mindset (M1), then plan and prevent (M2) before the build begins.



Modules 1 & 2 establish the Plan foundation, quality embedded before code (Shift-Left).



MODULE 01 · FOUNDATION

The Quality Mindset

Understand why quality is everyone's responsibility, and where SQA fits in the SDLC & Agile.

TIME

09:30 - 11:00

DURATION

1h 30m

THEME

Quality = everyone

01

What you'll master

After this module you can explain SQA's role across the SDLC & Agile, and tell apart the core terms people most often confuse.

01 Software quality basics

03 Verification vs Validation

05 SDLC vs STLC

07 Continuous Quality

02 QA vs QC

04 Testing vs Inspection

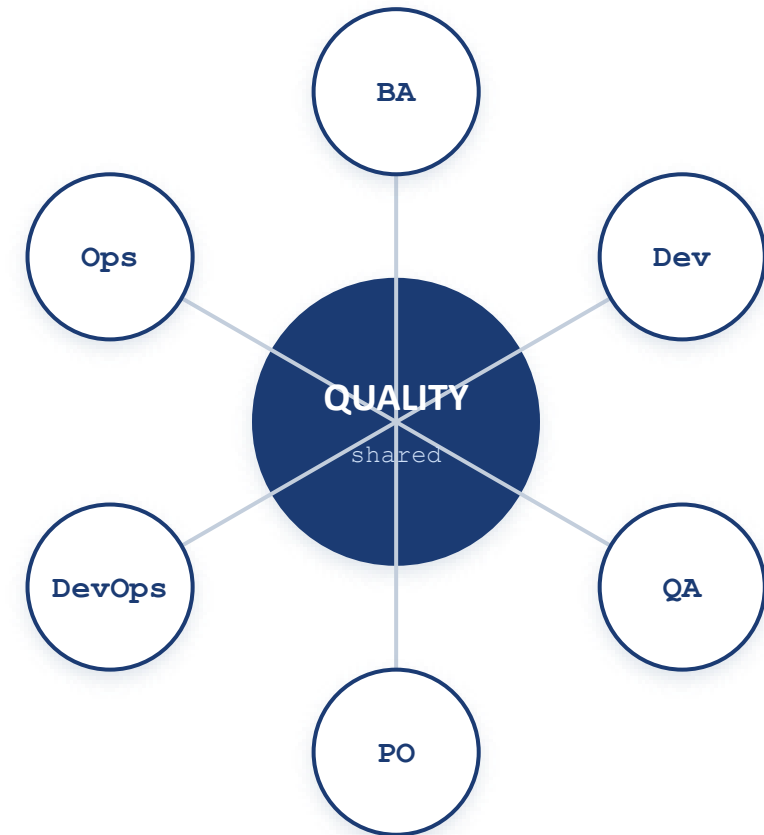
06 Shift-Left / Shift-Right

08 DevOps Quality

◇ Every topic is practiced on the ShopFast project, not theory alone.

Quality is everyone's job

- Quality isn't a final phase, it's embedded from requirements through operations.
- Every role contributes: BA clarifies requirements, Dev builds with tests, QA steers the process.
- SQA governs the process; it does not replace the team's ownership of quality.



QA vs QC

QA, Quality Assurance

“Are we doing things the right way?”

FOCUS	Process & prevention
WHEN	Throughout the SDLC
NATURE	Proactive · prevent defects
EXAMPLE	Standards, reviews, quality gates

VS

QC, Quality Control

“Does the product meet requirements?”

FOCUS	Product & detection
WHEN	After it’s built
NATURE	Reactive · find defects
EXAMPLE	Executing test cases

Verification vs Validation

Verification

"Are we building the product right?"

AGAINST

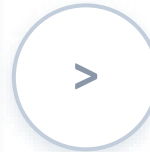
Spec & design

HOW

Reviews, walkthroughs (static)

ASKS

Conforms to the documents?



Validation

"Are we building the right product?"

AGAINST

Real user needs

HOW

Executing software (dynamic)

ASKS

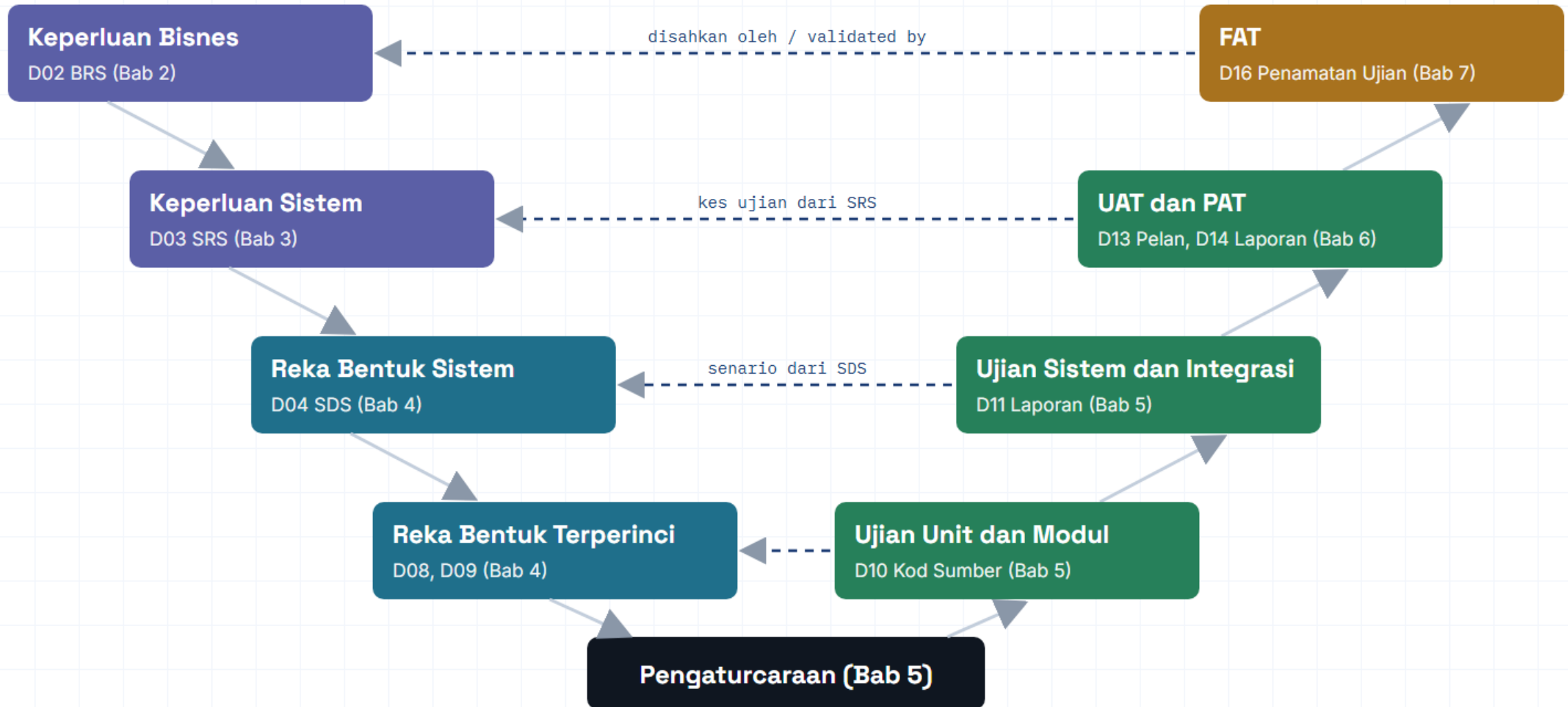
Solves the real problem?

◇ Verification catches misinterpretation early; Validation confirms it's actually useful.

Model V: setiap dokumen ada ujian yang membuktikannya

V-model: every document has a test that proves it

Peringkat ujian KRISAv2: Bab 5 (Unit, Integrasi, Sistem) dan Bab 6 hingga 7 (UAT, PAT, FAT)



Verifikasi: membina sistem dengan betul

Sumber: KRISAv2 Beta 2026 Bab 1 (Rajah 1.11, 1.12), Bab 5 5.7, Bab 6 Jadual 6.2, Bab 7 7.6; nombor templat ikut fail D01

Module hub

Open: KRISAv2 Bab 6 p.5

Dynamic vs static · process vs process



Testing vs Inspection

Testing, run the software and observe its behaviour (*dynamic*).

Inspection, review artifacts without executing them (*static*).

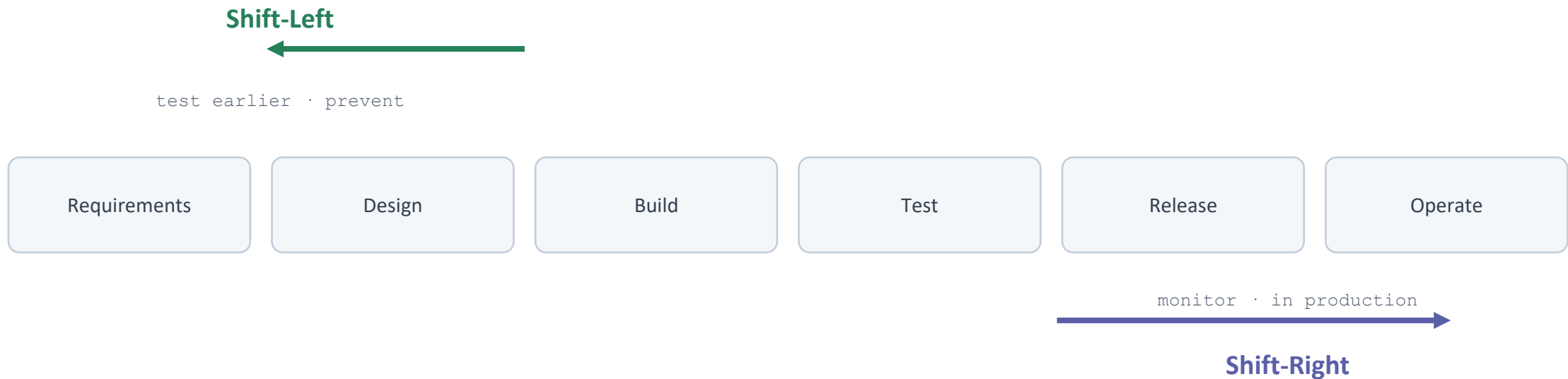


SDLC vs STLC

SDLC, the life cycle to build software: Requirements to Design to Build to Test to Deploy.

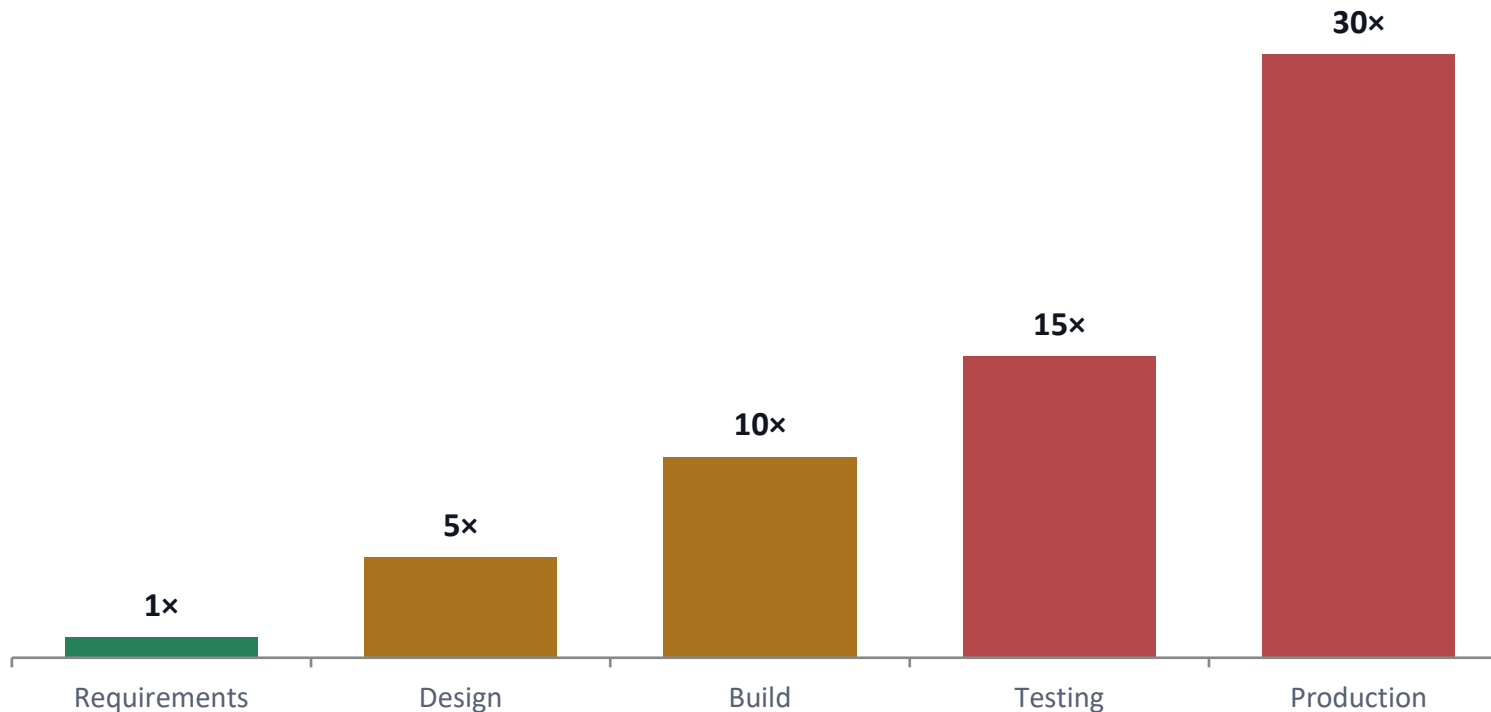
STLC, the testing life cycle running in parallel: analyse to plan to design to execute to close.

Shift-Left & Shift-Right



Shift-Left moves quality activities earlier so defects are caught when cheapest to fix. **Shift-Right** continues quality into production via monitoring & observability.

Cost to fix **escalates** the later you catch it



Prevent early

A defect caught in requirements can cost ~1x to fix; the same defect in production can cost many times more.

That is the economic case for Shift-Left.

◇ Relative, illustrative figures, the escalation pattern is a widely-cited idea (Boehm; IBM Systems Sciences Institute). Exact numbers vary by study.

Module 1 deep dive: the quality mindset

The general SQA ideas and vocabulary behind every review, test and audit in this workshop

01 Why software is hard

02 What quality means

03 Quality attributes

04 SQA, SQP, SQC and SQM

05 Fault, error and failure

06 Verification and validation

07 Testing philosophy

08 SQA glossary

Why software engineering is hard

Leveson's reasons software resists the discipline that physical engineering takes for granted

Curse of flexibility

No physical limits enforce discipline. Work starts before the need is understood, and one more feature always looks easy to add.

Organized complexity

Too interconnected to analyse part by part, too structured for averages. The most important properties emerge from interactions.

Intangibility

Software cannot be seen or touched, so progress, interfaces and design flaws stay invisible until something fails.

Large discrete state spaces

Behaviour is not continuous: one changed value can jump to a completely different state, so exhaustive testing is impossible.

No historical usage data

Each system is specially built and usually does something new, so there are few proven standard designs to measure and improve.

ShopFast view

Quantity, promo code, membership, payment method and order status combine into far more states than anyone can check by hand.

Historical project failure data: read with care

Figures from the 1990s quoted in Leveson's notes, which she herself headed 'Data (Myths?)'. Treat them as historical, not current

Claim reported	Figure	Reported by
Failure or cancellation rate of large software systems	Over 20 percent	Capers Jones
Systems over 100,000 LOC cancelled before completion	About 25 percent	Capers Jones
Systems over 500,000 LOC cancelled before completion	About 50 percent	Capers Jones
Systems over 1,000,000 LOC cancelled before completion	About 65 percent	Capers Jones
IT projects cancelled, from a survey of 8,000 projects	About 30 percent	Standish Group
Average cancelled US project	About a year late, having used 200 percent of expected budget	Capers Jones
Completed projects with schedule delays and cost overruns	About two thirds	Capers Jones
Completed projects with low reliability in the first year	About two thirds	Capers Jones

Source: Leveson, MIT 16.355 notes (1999), citing Capers Jones and the Standish Group

What is software quality?

Software quality

Conformance to explicitly stated functional and performance requirements, explicitly documented development standards, and implicit characteristics that are expected of all professionally developed software. (Pressman)

Requirements are the foundation: lack of conformance to requirements is lack of quality.

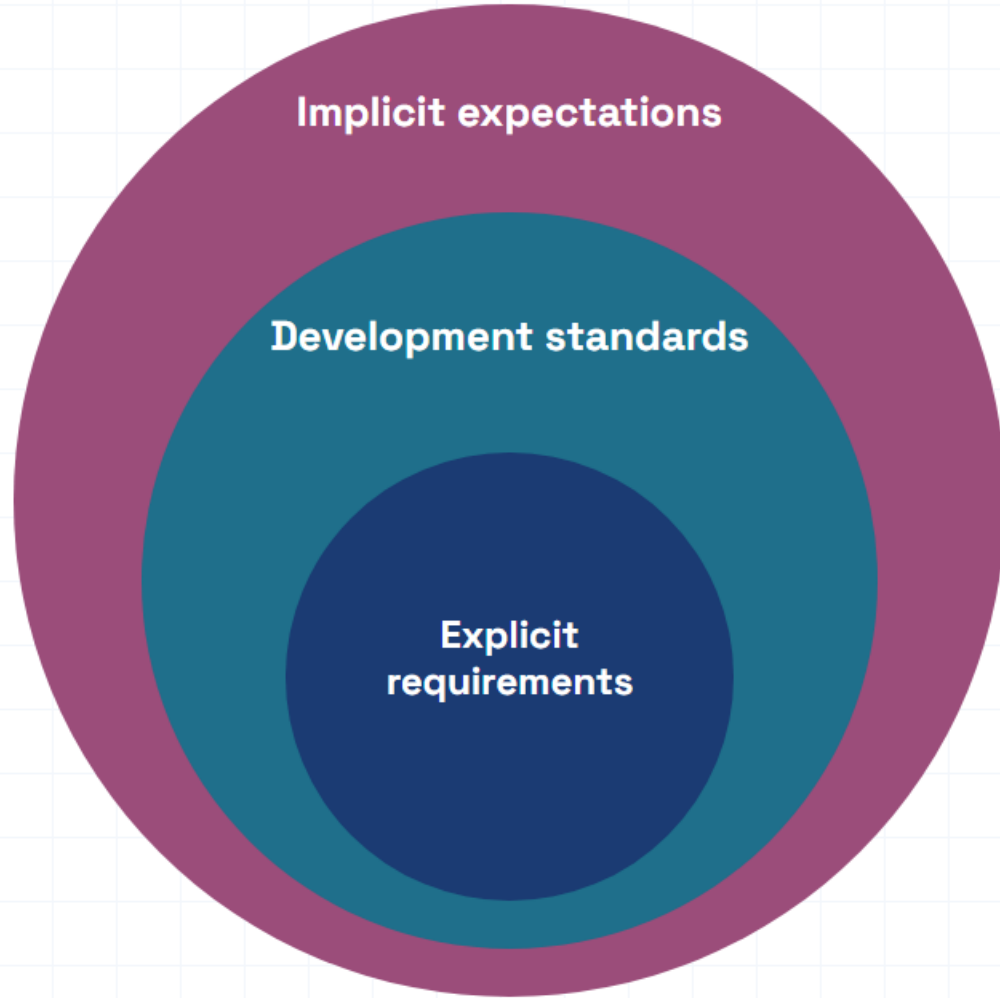
Standards guide how software is engineered; if their criteria are not met, poor quality almost surely follows.

'Meets the spec' is too simple: specifications are often incomplete, ambiguous, or silent on maintainability.

Users left out of requirements work may reject a product that conforms to every written line.

Three layers of software quality

Quality is conformance to all three layers, not only the written specification



Explicit requirements

Stated functional and performance requirements. ShopFast: quantity 1 to 10, free shipping at RM200.

Development standards

Documented criteria for how software is engineered: coding standards, KRISA templates, review rules.

Implicit expectations

Unwritten but expected of all professional software: no crashes, sensible errors, secure handling of data.

Quality attributes and their tensions

Fifteen attributes from the source grouped into three families, plus the static and dynamic view

Dependability

Safety, security, reliability, resilience, robustness. Does ShopFast keep working, and fail safely, when things go wrong?

Understand and change

Understandability, testability, adaptability, modularity, low complexity. Can the team safely change promo rules next year?

Use and reach

Usability, learnability, efficiency, portability, reusability. Can a first-time buyer check out quickly on any phone?

Static attributes

Belong to code and documents: well structured, maintainable and testable code; correct and complete documentation.

Dynamic attributes

Seen while the system runs: reliability, correctness, completeness, consistency, usability and performance.

Tensions

Improving one attribute can cost another: efficiency vs reliability is the source example; extra security steps can hurt usability.

Quality vs quality management

Quality is a property of the product; quality management is the organised work that makes it achievable

Quality

- Simplest definition: the developed product meets its specification.
- Problem: the organisation may demand more than the customer specified, which adds cost.
- Problem: some characteristics, such as maintainability, cannot be specified unambiguously.
- Problem: users not involved in requirements may not see a conforming product as quality.

Quality management

- Ensures the required level of product quality is actually achieved.
- Defines procedures and standards, and applies them to the product and the process.
- Checks that the procedures are followed.
- Collects and analyses quality data.
- Limit: intangible aspects such as elegance and readability cannot be standardised.

You cannot inspect quality into ShopFast at the end. Quality management builds the conditions for quality from day one.

The SQA framework: four elements

In the broad sense SQA includes all four; each answers a different question

SQA

Software Quality Assurance

Organisation-wide network of procedures and standards that lead to high-quality software

What are our rules?

SQP

Software Quality Planning

Select and tailor those procedures and standards for one specific project

Which rules apply here?

SQC

Software Quality Control

Ensure the project team actually follows the procedures and standards

Are we following them?

SQM

Software Quality Metrics

Collect and analyse quality data to predict and control product quality

Is quality improving?

ShopFast: the agency QA manual (SQA), the ShopFast QA Plan (SQP), review and audit records (SQC), defect density and pass rate trends (SQM).

SQA, SQP, SQC and SQM compared

Element	Definition (paraphrased)	Level	ShopFast example
SQA: Quality Assurance	Network of organisational procedures and standards that lead to high quality software	Organisation	Agency review procedure, coding standard and test document templates
SQP: Quality Planning	Selecting suitable procedures and standards and adapting them to one project	Project, at start	ShopFast quality plan naming which reviews, test levels and records apply
SQC: Quality Control	Defining and enacting checks that the team really follows project procedures	Project, ongoing	SRS review held, test logs kept, audit of change control before release
SQM: Quality Metrics	Collecting and analysing quality data to predict and control quality	Project and organisation	Open defects by severity, defect age, requirement changes per sprint

Seven SQA activities

What an SQA function actually does across the lifecycle

Apply technical methods

Use sound analysis and design methods so the specification and design are of high quality from the start.

Formal technical reviews

Structured meetings of technical staff whose sole purpose is to uncover quality problems.

Test the software

Use test case design methods that make error detection effective rather than random.

Enforce standards

Check that agreed development standards are actually followed on every work product.

Control change

Manage changes in development and maintenance so fixes do not silently break other parts.

Measure

Track quality, and judge whether changes to methods and procedures really improve it.

Keep records and report

Collect and share SQA information through defined procedures so the evidence survives.

Advantages and disadvantages of SQA

Advantages

- Fewer latent defects, so less effort and time spent in testing and maintenance.
- Higher reliability, which leads to greater customer satisfaction.
- Lower maintenance costs.
- Lower overall life cycle cost of the software.

Disadvantages

- Hard to institute in small organisations without spare people for the activities.
- It represents cultural change, and change is never easy.
- It needs money that would not otherwise be budgeted for engineering or QA.

The costs are visible and paid up front; the savings arrive later as defects that never reach users. Make that trade explicit for sponsors.

Fault, error and failure

- Fault: a static defect in the software, such as an incorrect, missing or extra instruction.
- Error: an incorrect internal program state that is the manifestation of some fault.
- Failure: external behaviour that is incorrect with respect to the requirements.
- numZero([2, 7, 0]) returns 1, which is correct: the fault is reached and the state is wrong, yet no failure.
- numZero([0, 7, 2]) returns 0 instead of 1: the error propagates to the output, so a failure occurs.

NUMZERO: COUNT ZEROS IN AN ARRAY (JAVA)

```
public static int numZero(int[] x) {  
    // effects: if x == null throw NullPointerException  
    //           else return the number of occurrences of 0 in x  
    int count = 0;  
    for (int i = 1; i < x.length; i++) { // fault: should start at 0  
        if (x[i] == 0) {  
            count++;  
        }  
    }  
    return count;  
}
```

Fault, error, failure: how a defect becomes visible

Three conditions must all hold before a user observes a failure (the RIP model)

Fault

A static defect in the code or document

```
for (int i = 1; i < x.length; i++)  
  should start at i = 0
```

then

Error

An incorrect internal state while running

```
numZero([0, 7, 2]): the first  
element is skipped, count  
stays 0
```

then

Failure

Incorrect external behaviour against the requirement

```
returns 0, expected 1
```

Reachability

The faulty location must be executed

Infection

The program state must become incorrect

Propagation

The bad state must reach an output

`numZero([2, 7, 0])` reaches the fault and the state is wrong (`i` starts at 1), but that wrong state never reaches the output: count is still 1. Error without failure, so a passing test does not prove there is no fault.

Terminology: mistake, fault, defect, bug, error, failure

Sources use these words differently. Agree one vocabulary for your project and record it in the quality plan

Term	Meaning	Used in	ShopFast example
Mistake (human error)	A human action that produces an incorrect result	ISTQB and IEEE glossaries, where it is called error	Analyst misreads the half-up rounding rule
Fault	A static flaw in code or another work product that can cause a failure	Sommerville; Ammann and Offutt	Rounding routine written half-down instead of half-up
Defect	Common synonym for fault; ISTQB treats defect, bug and fault as one idea	ISTQB, defect tracking tools	Logged as a defect record with severity and priority
Bug	Informal word for a fault, or loosely for a failure report	Everyday practice	'The checkout bug' may mean the fault or the symptom, so clarify
Error (program state)	An incorrect internal state caused by a fault	Ammann and Offutt; intro lecture	Discount variable holds 13.48 instead of 13.49 before the total
Failure	Observable incorrect behaviour against requirements or expectations	All sources	Receipt total is 1 sen lower than the correct total
Incident or anomaly	An observed event needing investigation, not yet confirmed as a failure	IEEE 1044 (anomaly); older ISTQB (incident)	Tester reports an odd total; triage decides whether it is a failure

Testing vs debugging

Testing

- Finds inputs that cause the software to fail.
- Starts from requirements and expected results.
- Can be planned, repeated and done by independent testers.
- Output: a failure report with steps, data, expected and actual results.

Debugging

- Finds the fault, given a failure.
- Starts from the failure and works back through code and program state.
- Usually done by the developer who owns the code.
- Output: a located and fixed fault, followed by a retest by the tester.

In practice they alternate in a cycle (Module 4 shows the test and debug cycle), but they are different jobs needing different skills.

Verification vs validation

Two sources, one idea: did we build the product right, and did we build the right product?

Verification

- Lecture: ensure the software product complies with its design.
- ISO 9000 wording: confirm, with objective evidence, that specified requirements are fulfilled.
- Boehm's short form: are we building the product right?
- ShopFast: a design review checks that the SDS matches the SRS discount rules.

Validation

- Lecture: ensure the software product complies with its intended usage.
- ISO 9000 wording: confirm requirements for a specific intended use are fulfilled.
- Boehm's short form: are we building the right product?
- ShopFast: agency users run real checkouts in UAT and confirm it suits their buyers.

Validation is usually harder, because intended use is rarely written as precisely as a design. KRISA UAT is mainly a validation activity.

Static vs dynamic testing

Static testing

- Examines work products without executing the program.
- Code walkthroughs, inspections and static analysis tools.
- Works on requirements, designs and documents, not just code.
- Finds faults directly, often before any code runs.

Dynamic testing

- Executes the program with real inputs and observes behaviour.
- Unit, integration, system and acceptance testing.
- Needs running code and a suitable environment.
- Reveals failures; debugging then locates the fault.

Use both. Static information, such as which code paths exist, often makes dynamic testing more efficient.

Testability and design for testability

Testability

The degree to which a system or component facilitates the establishment of test criteria and the performance of tests to determine whether those criteria have been met.

The more complex the application, the lower its testability and the more effort testing needs.

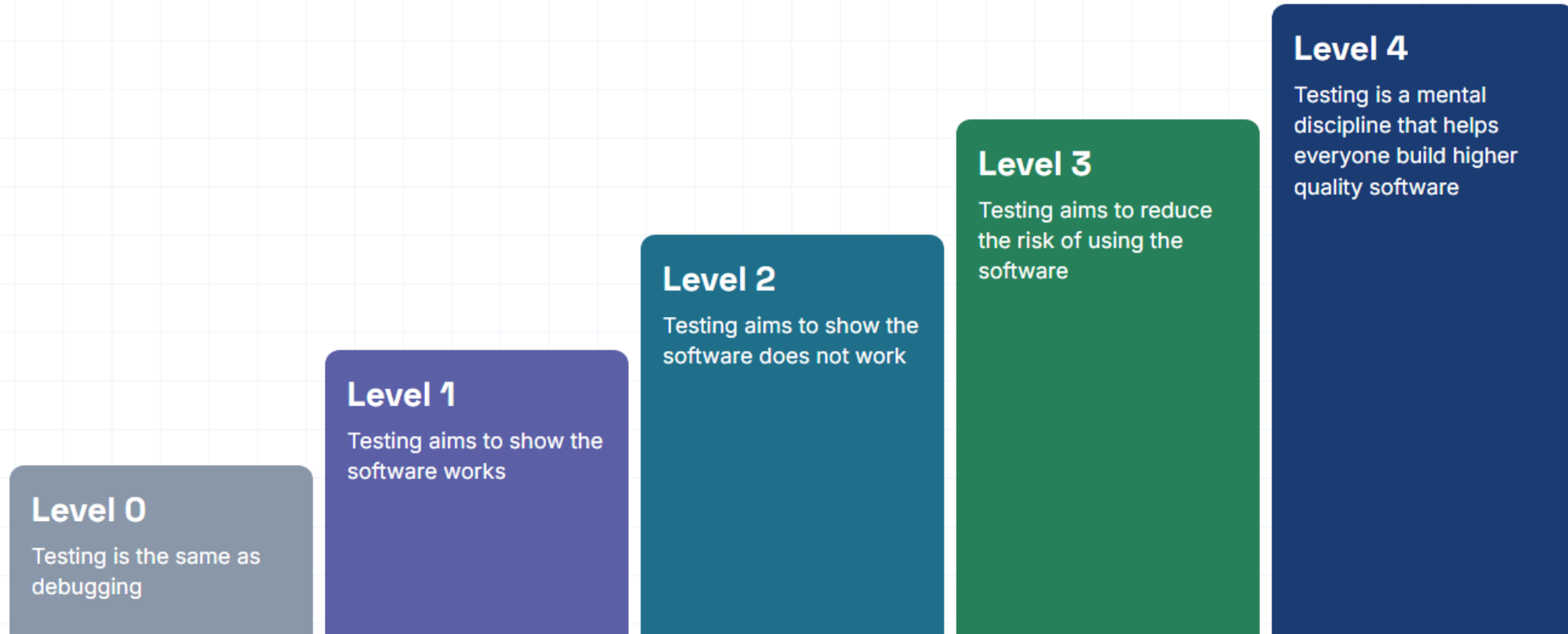
Design for testability: build software so it can be tested easily, not as an afterthought.

ShopFast: keep discount and shipping maths in a pure function tests can call without a browser.

ShopFast: let tests control the clock, so promo code expiry can be checked on any day.

Test process maturity: how a team thinks about testing

Beizer's levels describe the purpose a team gives to testing



Testing shows presence, not absence

“ *Program testing can be used to show the presence of bugs, but never to show their absence.* ”

Edsger W. Dijkstra (1970)

The lecture summary says the same: testing can show the presence of failures, not their absence.

A green ShopFast test run means no failures on those inputs, not a correct checkout.

So combine reviews, static analysis and risk-based testing, and report remaining risk honestly.

Exhaustive testing is impossible for real programs; Module 4 shows the path explosion.

Process quality vs product quality

Factory approach

- Assumes process quality directly determines product quality.
- Loop: define process, develop product, assess quality, then improve or standardise the process.
- Works well in manufacturing, where the same item is produced again and again.
- Limit: software is designed, not manufactured, so a good process alone does not guarantee a good product.

Creative approach

- Start by identifying products that turned out to be of good quality.
- Examine the processes used to develop those products.
- Generalise those processes so they can be applied everywhere.
- Keeps skilled people and design judgement in view, not only procedures.

Process matters, but it is not a vending machine. Improve process from evidence of what actually produced good products.

Quality is everyone's responsibility

“ *Quality is free, if it is everyone's responsibility.* ”

Tseytlin, Raytheon SQA course

Echoes Philip Crosby's book *Quality Is Free* (1979): the real cost lies in poor quality, not in doing it right.

Reviews make quality everyone's responsibility: it must be built in, not added by SQA at the end.

If only the SQA officer cares, quality becomes an expensive after-the-fact inspection.

ShopFast: analysts write testable criteria, developers write unit tests, SQA checks the system works.

SQA glossary (1 of 2): quality and management terms

Term	Plain meaning	ShopFast example
Software quality	Conformance to explicit requirements, development standards and implicit expectations	Checkout meets the REQ-CHK rules and never loses a paid order
Quality management	Defining, applying and checking procedures and standards, and analysing quality data	Agency process for reviews, testing and records
SQA	Organisation-wide framework of procedures and standards for high quality software	Agency SQA procedures used by every project
Software Quality Plan (SQP)	Project selection and tailoring of those standards, written for SQA and the customer	ShopFast quality plan
SQC	Checking that the project really follows its procedures and standards	Audit that every change has an approved request
SQM	Collecting and analysing quality data to predict and control quality	Defects found per review; open defects by severity
Process standard	A standard that defines how development work must be carried out	Review procedure; change control procedure
Product standard	A standard that applies to the product being developed	Coding standard; SRS template; UI style guide
Formal technical review	Structured meeting of technical staff to uncover quality problems in a work product	SRS review with a recorder and an issues list
Audit	Independent check that work complies with process standards	SQA audit of ShopFast configuration control

SQA glossary (2 of 2): defect and testing terms

Term	Plain meaning	ShopFast example
Fault (defect)	A static flaw in code or a document that can cause a failure	Rounding routine uses half-down instead of half-up
Error (program state)	An incorrect internal state caused by a fault	Discount holds 13.48 instead of 13.49
Failure	Observed behaviour that differs from requirements or expectations	Receipt total is 1 sen too low
Verification	Confirming a product complies with its design and specified requirements	Design review confirms the SDS matches SRS discount rules
Validation	Confirming the product suits its intended use	Agency users accept checkout in UAT
Static testing	Examining work products without running the program	Inspection of the discount module code
Dynamic testing	Running the program with inputs and observing its behaviour	Checkout run with quantity 3 and promo code RAYA15
Test case	Input data plus a precise description of the expected result	Quantity 11 is rejected with VALIDATION_ERROR
Test oracle	A mechanism that decides whether observed behaviour matches expected behaviour	Independent spreadsheet of correct totals for each test cart
Testability	How easily test criteria can be set and tests performed	Pricing logic in a pure function that tests call directly

Practice: where quality begins

1

In groups, list **3 real software failures** & their quality root causes.

2

Sketch the “**Cost of Poor Quality**”, the cost of fixing a bug at each SDLC phase.

3

Mark where **Shift-Left** could catch each defect earlier, using the ShopFast project.

What you carry out



OUTPUT to

A short “where quality begins” map for ShopFast + a **team glossary** for QA-vs-QC / V&V.



FEEDS INTO

Module 2

This glossary & prevention mindset becomes the base for the QA Plan and Requirement Review.



PRINCIPLE

Prevent, don’t just find

Every defect avoided early saves multiplied cost later, the core of Shift-Left.

Abbreviations used in this module (1 of 6)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
AI	Artificial Intelligence	Software that can answer questions and write text or code by learning patterns from large amounts of data.
API	Application Programming Interface	A doorway that lets one program ask another program for data or an action, without using a screen.
APM	Application Performance Monitoring	Tools that watch a live system and show how fast and healthy it is.
BA	Business Analyst	The person who gathers and clarifies what the business needs from the system.
BRS	Business Requirements Specification	The document that states what the business needs, written in business language. KRISA deliverable D02.
BVA	Boundary Value Analysis	Testing at the edges of the allowed values, such as 0, 1, 10 and 11, because mistakes often hide there.
CI	Continuous Integration, or Configuration Item	Two meanings. In DevOps, every code change is built and tested automatically. In configuration management, it is any controlled item such as a document or code file.
CI/CD	Continuous Integration and Continuous Delivery	An automated pipeline that builds, tests and prepares every change for release.

Abbreviations used in this module (2 of 6)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
DoD	Definition of Done	The agreed checklist a piece of work must pass before the team calls it finished.
EP	Equivalence Partitioning	Grouping inputs that behave the same way, then testing one value from each group.
FAT	Factory Acceptance Test	The formal test with the customer before the system is installed at the live site.
IBM	International Business Machines	A large technology company. Its research is often quoted on the cost of fixing defects late.
ID	Identifier	A unique label, such as REQ-CHK-01, so that each item can be traced.
IEEE	Institute of Electrical and Electronics Engineers	A professional body that publishes software engineering standards such as IEEE 730 and IEEE 829.
ISO	International Organization for Standardization	Publishes international standards such as ISO 9001 and ISO/IEC 25010.
ISTQB	International Software Testing Qualifications Board	Offers the best-known certifications for software testers.

Abbreviations used in this module (3 of 6)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
IT	Information Technology	Computers and software used to run an organisation's work.
IV&V	Independent Verification and Validation	Checks by a party outside the development team that the system is built right and is the right system.
JDN	Jabatan Digital Negara	The Malaysian government department that publishes KRISA and PPrISA.
KRISA	Kejuruteraan Sistem Aplikasi Sektor Awam	The Malaysian public sector guide for engineering application systems. KRISAv2 is the 2026 beta edition.
LOC	Lines of Code	The number of program lines. A rough measure of size.
MIT	Massachusetts Institute of Technology	A university in the United States. Source of the Leveson course notes.
PAT	Provisional Acceptance Test	The KRISA acceptance test done before provisional acceptance of the system, including performance checks.
PDF	Portable Document Format	A document file that looks the same on every computer.

Abbreviations used in this module (4 of 6)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
PO	Product Owner	The person who decides what the team builds next and accepts the result.
PPrISA	Panduan Pengurusan Projek ICT Sektor Awam	The Malaysian public sector guide for managing ICT projects. Codes such as PPrISA05 are its templates.
PPS	Pelan Pembangunan Sistem	The System Development Plan. KRISA deliverable D01.
PS	PowerShell prompt	When a slide shows PS>, type that command in Windows PowerShell.
QA	Quality Assurance	Preventing defects by setting up good processes and checking that people follow them.
QC	Quality Control	Finding defects in the product itself through testing and inspection.
REQ	Requirement	Prefix in requirement IDs such as REQ-CHK-01. CHK stands for checkout.
RIP	Reachability, Infection, Propagation	The three conditions for a fault to become a visible failure: the faulty code runs, it corrupts the program state, and the wrong state reaches the output.

Abbreviations used in this module (5 of 6)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
RM	Ringgit Malaysia	Malaysian currency.
RTM	Requirements Traceability Matrix	A table linking each requirement to its design, tests and defects, so nothing is missed.
SDLC	Software Development Life Cycle	The phases a system goes through, from planning to maintenance.
SDS	Spesifikasi Reka Bentuk Sistem	The System Design Specification. KRISA deliverable D04.
SPA	Security Posture Assessment	A security check of the system and its setup before it goes live. A KRISA and PPrISA activity.
SQA	Software Quality Assurance	All the planned work that gives confidence the software meets its quality requirements.
SQC	Software Quality Control	Checks on the product itself: reviews, tests and audits.

Abbreviations used in this module (6 of 6)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
SQM	Software Quality Metrics	Numbers used to measure quality, such as defect density.
SQP	Software Quality Plan	The plan stating which standards, reviews, tests and audits a project will use.
SRS	Spesifikasi Keperluan Sistem	The System (or Software) Requirements Specification: what the system must do. KRISA deliverable D03.
STLC	Software Testing Life Cycle	The phases of testing, from planning to closure.
UAT	User Acceptance Test	Users try the system to confirm it meets their needs before they accept it.
UI	User Interface	The screens, buttons and forms that people use.
V&V	Verification and Validation	Verification asks: did we build it right? Validation asks: did we build the right thing?

References and templates for this module

Every line opens the official JDN document at the exact page. Templates download as Word or PDF.

WHERE IT IS IN THE GUIDES

- Bab 1, 1.5 Metodologi KRISA (6 fasa)
page 11 · KRISAv2 Bab 1 Perancangan
- Bab 1, 1.6.4 Jaminan Kualiti Perisian (SQA), Jadual 1.3 Atribut Kualiti
page 19 · KRISAv2 Bab 1 Perancangan
- Bab 1, Rajah 1.11 dan 1.12 Verifikasi dan Validasi
page 20 · KRISAv2 Bab 1 Perancangan
- Bab 1, Jadual 1.4 Jenis Pengujian IV&V
page 22 · KRISAv2 Bab 1 Perancangan
- Bab 8, KRISA + PPrISA + DevOps saling melengkapi
page 1 · KRISAv2 Bab 8 Penyelarasan DevOps
- PPrISA 2.0, 4.2.1 b) skop: SPA dan Independent Validation & Verification
page 60 · PPrISA 2.0 Versi Beta Feb 2025
- PPrISA 2.0, peranan pasukan Jaminan Kualiti dan Pengujian Sistem
page 53 · PPrISA 2.0 Versi Beta Feb 2025

TEMPLATES TO USE

- D01
Pelan Pembangunan Sistem (PPS)

WORD

PDF
- PPrISA02
Pelan Pengurusan Projek

WORD

PDF

M01 · W01 · 20 MINUTES · Hari 1, 10:35 hingga 10:55

Hands-On SQA-AI Assistant Workshop

AI as your quality coach

Use the read-only Plan agent to analyse five ShopFast incidents, see where each defect was injected, then let the Build agent save a bilingual QA glossary.

TOOL

opencode 1.18 in the lab folder · model: Big Pickle via OpenCode Zen (free, no API key) · synthetic ShopFast data only

[Step-by-step guide \(web\)](#)[Workshop slides \(W01\)](#)[Setup guide W00](#)[opencode docs](#)

AI as your quality coach

STEPS

- 1 Start opencode in Plan mode and analyse the incidents
- 2 Ask for a Cost of Poor Quality table in the same session
- 3 Switch to Build and ask for the glossary file
- 4 Approve the file write and open the result

OUTPUT TO SAVE

outputs/m01-glosari-qa.md

TYPE THIS (opencode> in opencode, PS> in PowerShell)

```
opencode> Baca @shopfast/docs/incidents.md. Bagi setiap insiden berikan:
punca akar, fasa SDLC kecacatan disuntik, fasa ia ditemui, dan aktiviti SQA
yang boleh menangkapnya lebih awal. Rujuk KRISAv2 Bab 1 1.6.4 menggunakan
jadual rujukan dalam AGENTS.md. Jadual maksimum 6 lajur.
```

```
opencode> Bina jadual Cost of Poor Quality (fasa, kos relatif, contoh
ShopFast) dan terangkan Shift-Left dalam 3 ayat.
```

```
opencode> Tulis fail outputs/m01-glosari-qa.md: 12 istilah (QA, QC,
Verifikasi, Validasi, Ujian Statik, Shift-Left, Shift-Right, Quality Gate,
Kecacatan, Kegagalan, IV&V, DevOps) dalam satu jadual dengan takrif Bahasa
Melayu dan Inggeris.
```

HUMAN CHECK BEFORE YOU ACCEPT IT

- Every incident is mapped to a real KRISAv2 phase (Permulaan to Pelaksanaan)
- QA is described as prevention and QC as detection
- Links point to the KRISAv2 page table, not invented page numbers

Open step-by-step guide

Module hub

Pembantu AI dalam SQA: AI mendraf, manusia bertanggungjawab

AI assistant in SQA: AI drafts, humans remain accountable

Aliran kerja setiap bengkel Hands-On SQA-AI Assistant Workshop menggunakan opencode

1

Konteks

@SRS, @templat,
AGENTS.md

2

Prompt

arahan /review-srs

3

Draf AI

opencode + model

4

Semak manusia

banding dengan templat
KRISA/PPrISA

5

Simpan bukti

outputs/ + git commit

Guna AI untuk

Draf pelan dan laporan, cadang kes ujian
EP/BVA, jana skrip ujian, ringkaskan log, cari
jurang RTM.

Jangan

Tampal data sebenar agensi, terima output
tanpa semak, biar AI reka ID keperluan, kongsi
sesi awam.

Bukti audit

Simpan prompt, output dan semakan dalam git
supaya jejak audit lengkap (Modul 7).



MODULE 02 · PLAN

02

QA Planning & Proactive Quality

Build quality before coding begins, produce a QA Plan, review requirements, and start the RTM.

TIME

11:15 - 13:00

DURATION

1h 45m

DELIVERABLE

QA Plan · RTM v0

Jadual slot: Modul 2: Perancangan QA dan Kualiti Proaktif

Run sheet / session timing

Hari 1, 11:15 hingga 13:00 · 105 minit · kira-kira 80% amali



Masa	Segmen	Tempoh
11:15 hingga 11:45	Konsep	30 minit
11:45 hingga 12:30	Makmal	45 minit
12:30 hingga 12:55	Bengkel AI W02	25 minit
12:55 hingga 13:00	Rumusan	5 minit

Quality is built, not tested in

This module's focus: set up the process, criteria and traceability so defects are prevented before code exists.

01 Test Strategy

03 Definition of Ready / Done

05 Risk Register

07 Static Testing

09 BRS · SRS (KRISA)

02 Test Plan

04 Quality Gates

06 Acceptance Criteria (INVEST)

08 Function Points (sizing)

10 RTM · Gap Analysis

◇ Every topic is practiced on the ShopFast project, not theory alone.

From understanding to planning



Set the process

Test Strategy & Test Plan define scope, approach and success criteria.



Review requirements

Static testing catches vague & untestable requirements before they become code.



Trace everything

The RTM links every requirement to test case to defect.

◇ This is Shift-Left in practice: quality embedded before the build.

Test Strategy **vs** Test Plan

Test Strategy

Organisation / product level, the direction.

SCOPE Overall principles & approach

STABILITY Rarely changes

ANSWERS How do we test in general?



Test Plan

Project / feature level, the execution.

SCOPE Scope, entry/exit, schedule, resources

STABILITY Specific per release

ANSWERS What do we test for ShopFast?

Definition of Ready to Done to Gate



Definition of Ready, conditions to start (clear, testable, has acceptance criteria). **Definition of Done**, conditions to be truly finished. A **Quality Gate** enforces it.

INVEST + Given / When / Then

INVEST

- I** Independent
- N** Negotiable
- V** Valuable
- E** Estimable
- S** Small
- T** Testable

EXAMPLE · SHOPFAST CHECKOUT

Given a cart with items and a signed-in user

When the user taps “Pay”

Then the order is confirmed & a receipt is shown

Criteria written like this convert straight into test cases.

Catch **flawed requirements** early

Static testing reads the document without running code, hunting ambiguity, contradictions and untestable requirements in a deliberately flawed SRS.



"system must be fast"

■ no measurable threshold



"user-friendly interface"

■ subjective · untestable



"handle many users"

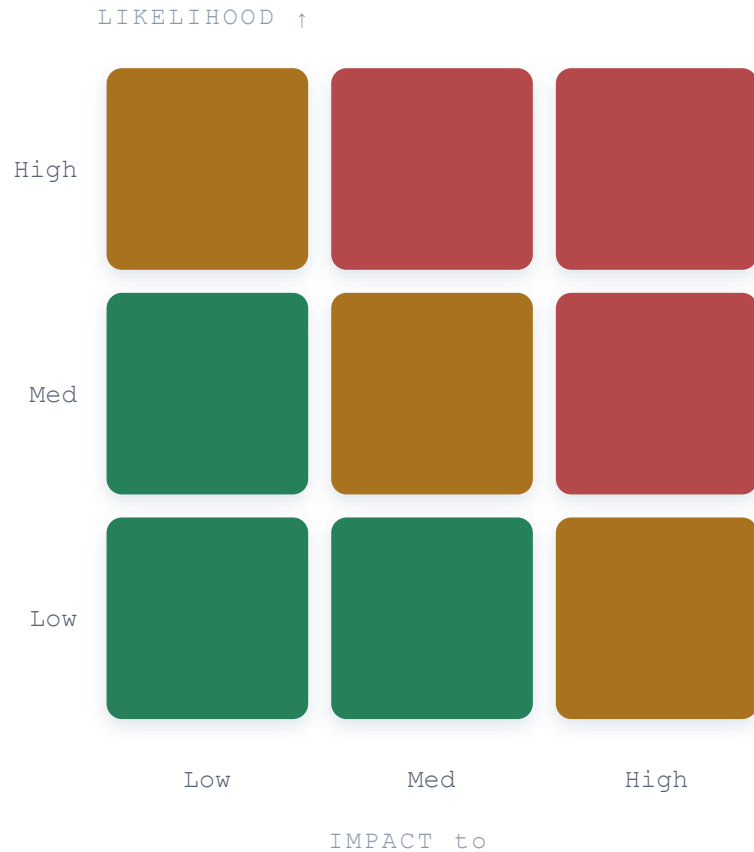
■ how many? no number



"etc."

■ scope left open

Risk Register & risk matrix

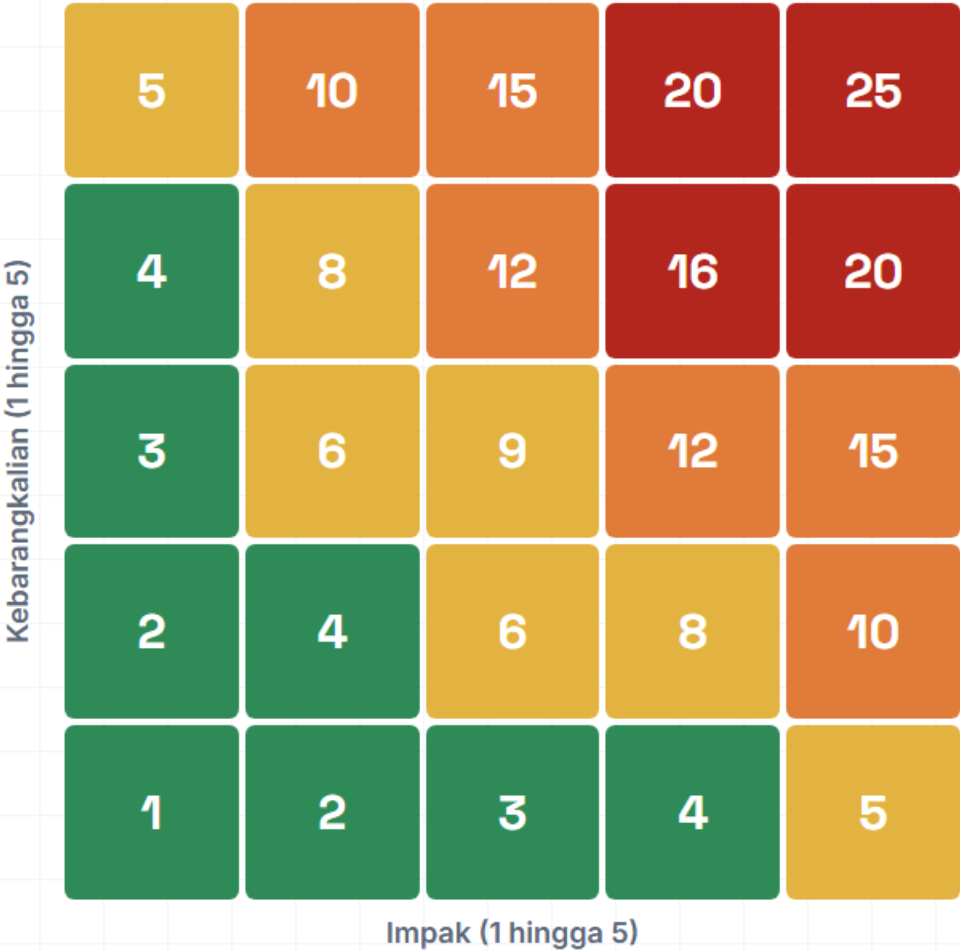


- Each risk is scored: likelihood × impact.
- Red cells = test first and most thoroughly; green = light coverage.
- For ShopFast, payment & security are usually the highest risks.
- The Register makes coverage decisions transparent & defensible.

Matriks risiko 5 x 5: uji mengikut risiko

5 x 5 risk matrix: test by risk

Skor = Kebarangkalian x Impak. Merah dan jingga diuji dahulu dan paling mendalam. Daftar dalam templat PPrISA07.



ID	Risiko (contoh ShopFast)	K	I	Skor
R1	Order boleh dilihat tanpa semakan pemilik (IDOR)	4	5	20
R2	Diskaun salah kira semasa jualan perayaan	4	4	16
R3	Had kuantiti troli tidak dikuatkuasa	3	3	9
R4	Log merekod emel dan telefon (PDPA)	3	4	12
R5	Masa respons checkout perlahan	2	3	6

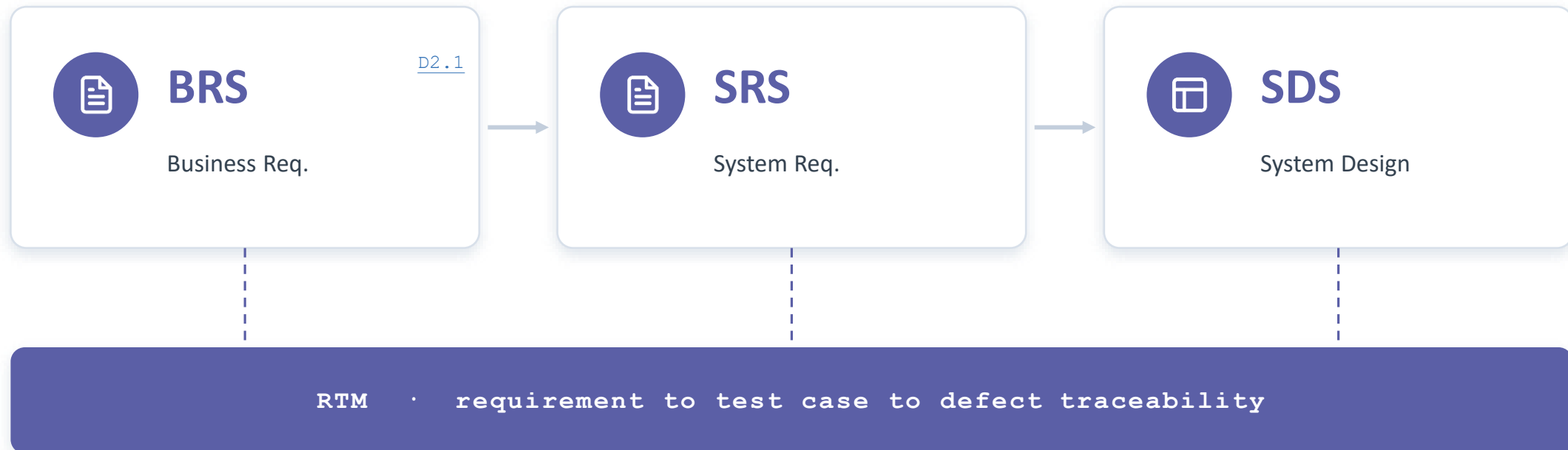
16 hingga 25 Kritikal

10 hingga 15 Tinggi

5 hingga 9 Sederhana

1 hingga 4 Rendah

BRS to SRS to SDS, traced by the RTM

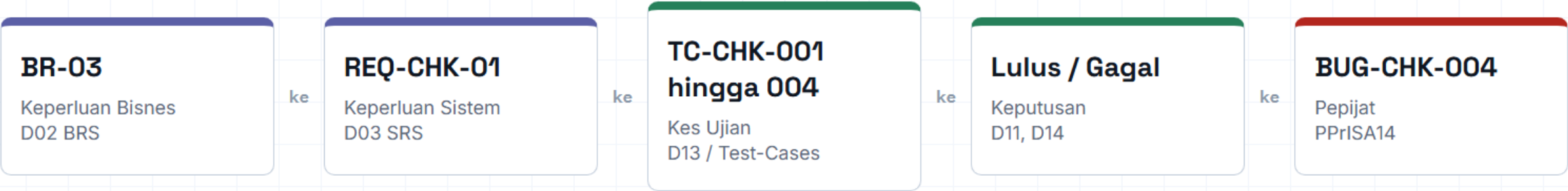


BRS / SRS capture business & system requirements; **Function Points** give an early size for estimation. The **RTM** ensures no requirement goes untested.

Matriks kebolehhesanan (RTM): dari keperluan ke bukti

Requirements Traceability Matrix: from requirement to evidence

Ke hadapan: setiap keperluan ada ujian. Ke belakang: setiap ujian dan pepijat ada keperluan. KRISAv2 Bab 2 2.11, Templat T2.8.



BR_ID	FR_ID	Keperluan	Test_Case_IDs	Peringkat	Status	Defect_IDs	Liputan
BR-03	REQ-CHK-01	Kuantiti setiap item 1 hingga 10	TC-CHK-001 hingga 004	Sistem	Gagal	BUG-CHK-004	100%
BR-04	REQ-CHK-03	Penghantaran percuma bagi subjumlah RM200 dan ke atas	TC-CHK-008 hingga 012	Sistem	Lulus		100%
BR-06	REQ-CHK-06	Checkout p95 bawah 2 saat, 50 pengguna serentak	(tiada)		Belum diuji		0% jurang

Module 2 deep dive: planning for quality

Standards, quality plans, lifecycle deliverables and review gates that turn quality intentions into a workable plan

01 Why standards matter

02 Process and product standards

03 ISO 9001 then and now

04 IEEE 730 and ISO/IEC/IEEE 90003

05 Quality plan hierarchy

06 Independence of SQA

07 Deliverables and review gates

08 Planning checklists

Why standards matter

Standards capture practice that has worked, so each project does not start from zero

- **Best practice, captured:** Standards encapsulate the best, or at least the most appropriate, practice the organisation has learned.
- **A framework for QA:** They give quality assurance something concrete to check against; without a standard, SQC has nothing to audit.
- **Continuity:** Work passes between analysts, developers, testers and maintainers; standards keep it consistent across the lifecycle.
- **Tailor, do not avoid:** If a standard is too extensive for the task, tailor it. Avoiding it altogether is not an option.
- **Many sources:** In mature organisations ISO is not the only source, and process and product standards are derived independently.
- **SQA does not write all:** Product standards, such as coding rules, are usually not created by SQA; engineering owns them.
- **Link to KRISA:** KRISAv2 and PPrISA templates act as the agency's standards; ShopFast tailors them rather than inventing its own.

Where software development standards come from

Process standards say how to work; product standards say what the result must look like

Process standards

how software must be developed

ISO 9001

ISO/IEC/IEEE 12207

CMMI

IEEE 730

then organisation quality manual and development process standards (for Malaysia: KRISA, PPrISA)

Project SDP

Project SQP

Project SCM plan

Method sheets

Product standards

what the product and documents must conform to

ISO/IEC 25010

Industry and MIL standards

COTS standards

OWASP ASVS

then organisation product standards (coding standards, document templates such as D01 to D18)

Project coding rules

Interface specs

Document formats

In mature organisations ISO is not the only source, process and product standards are derived independently, and product standards are owned by engineering, not by SQA.

Process standards vs product standards

Process standards

- Define how work is done during development.
- Sources: ISO 9001, CMMI, the organisation quality manual and process standards.
- Project level: development plan, SQP, SCM plan, method sheets.
- ShopFast: every SRS change needs an approved change request before coding starts.

Product standards

- Define what the product and its documents must look like.
- Sources: ISO, industry or military standards, COTS vendor rules, organisation standards.
- Project level: coding rules, document templates, interface formats.
- ShopFast: money kept to 2 decimals rounded half-up; the SRS follows the D03 template.

Check process standards by examining records of what people did; check product standards by examining the work product itself.

ISO 9001:1994: 20 elements as a software quality checklist

Tseytlin marks which of the 20 elements are software quality responsibilities. The 1994 structure is historical but still a useful checklist

Element (4.1 to 4.10)	Software quality role?	Element (4.11 to 4.20)	Software quality role?
4.1 Management responsibility	Yes	4.11 Inspection, measuring and test equipment	Not marked
4.2 Quality system	Yes	4.12 Inspection and test status	Yes
4.3 Contract review	Yes	4.13 Control of nonconforming product	Not marked
4.4 Design control	Yes	4.14 Corrective action	Yes
4.5 Document control	Yes	4.15 Handling, storage, packaging, delivery	Not marked
4.6 Purchasing	Yes	4.16 Quality records	Yes
4.7 Purchaser supplied product	Not marked	4.17 Internal quality audits	Yes
4.8 Product identification and traceability	Yes	4.18 Training	Yes
4.9 Process control	Yes	4.19 Servicing	Not marked
4.10 Inspection and testing	Yes	4.20 Statistical techniques	Yes

ISO 9001:2015: the current clause structure

Since 2000 a process approach replaced the 20 elements; the 2015 edition uses 10 clauses shared with other ISO management standards

Clause	Title	What it asks for	SQA view on ShopFast
1 to 3	Scope, normative references, terms	Framing only, no auditable requirements	Not audited
4	Context of the organisation	Understand issues, interested parties and the scope of the QMS	Who depends on ShopFast: agency, sellers, buyers
5	Leadership	Top management commitment, quality policy, roles and authorities	Sponsor endorses the policy; SQA reports to management
6	Planning	Address risks and opportunities, set quality objectives, plan changes	Risk register and quality objectives in the quality plan
7	Support	Resources, competence, awareness, communication, documented information	Trained testers; controlled templates and records
8	Operation	Operational control: requirements, design, suppliers, release, nonconforming outputs	SRS review, design review, UAT sign-off, release control
9	Performance evaluation	Monitoring, measurement, internal audit, management review	Defect metrics, internal audit, management review
10	Improvement	Nonconformity, corrective action, continual improvement	Root cause and corrective action after production defects

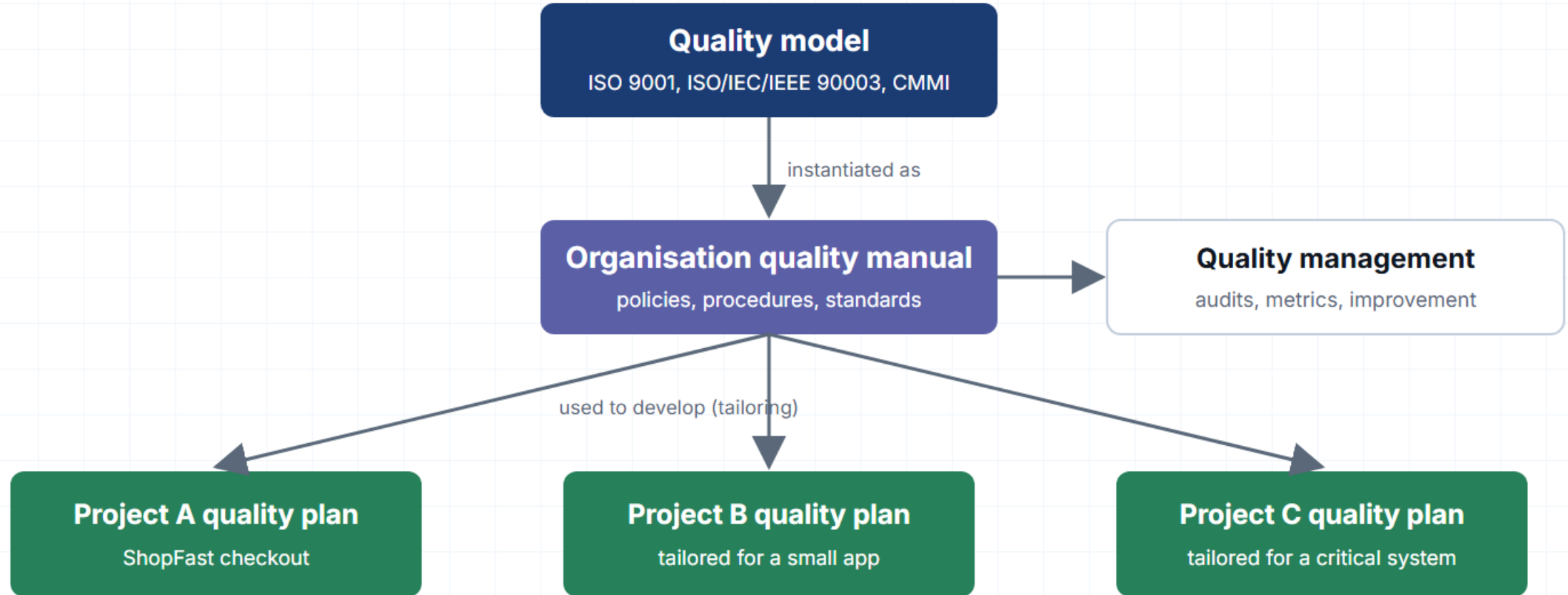
Standards for SQA planning: then and now

The source slides cite ISO 9000-3 and ANSI/IEEE standards. This is what those references mean today

Standard	What it covers	Status and use
IEEE 730	Software quality assurance processes, including what an SQA plan must address	Current edition IEEE 730-2014; main reference when writing an SQA plan
ISO 9000-3	Guidelines for applying ISO 9001 to software development and maintenance	Historical; replaced by ISO/IEC/IEEE 90003
ISO/IEC/IEEE 90003	Guidance for applying ISO 9001 to computer software	Current edition 2018; use when the agency runs an ISO 9001 system
ISO/IEC/IEEE 12207	Software life cycle processes, including quality assurance	Current edition 2017; names the processes a plan must cover
IEEE 829	Software test documentation	Superseded by ISO/IEC/IEEE 29119-3 for test plans and reports
ISO 9001	Quality management system requirements for any organisation	Current edition 2015; conformance verified by external certification bodies

From quality model to project quality plans

The organisation instantiates a model once; every project tailors it



Tailoring means selecting what fits the project. It never means having no standards.

What a Software Quality Plan contains

The SQP is the project's agreement about how quality will be achieved and checked, written for a specific audience

- **Tailoring:** Selects the organisation standards appropriate to this product, and records why others do not apply.
- **Standardisation:** Calls out only approved organisation process and product standards, by name and version.
- **New standard needed:** If no standard fits, initiate a quality improvement to create one rather than inventing it inside the plan.
- **Structure:** Elements are usually based on the ISO 9001 model; IEEE 730 gives a current checklist of plan content.
- **Audience:** Written for SQA engineers as a guide for SQC, and for the customer to monitor development; not for developers.
- **Boundaries:** Software production, product plans and risk management belong in the development plan (for KRISA, D01 PPS).
- **No quality trade-off:** Quality factors are not sacrificed for efficiency. If the quality process cannot be upheld, do not take the job.

Independence and continuity of SQA

Independence

- SQA must be independent to take an objective view of the process.
- It reports problems directly to senior management, not only to the project manager.
- Audit findings not fixed at project level are escalated upward (Module 7).
- ShopFast: the SQA officer does not report to the delivery lead.

Continuity

- Standards are upheld no matter how small the task.
- If a prescribed process does not suit the product, tailor it and record the tailoring.
- Prototyping does not mean no standards; it means tailored standards.
- ShopFast: a quick promo code prototype still gets a short review and a tracked change.

Independence gives SQA a voice that can be heard; continuity stops standards from disappearing whenever a deadline gets close.

Lifecycle deliverables vocabulary and KRISA equivalents

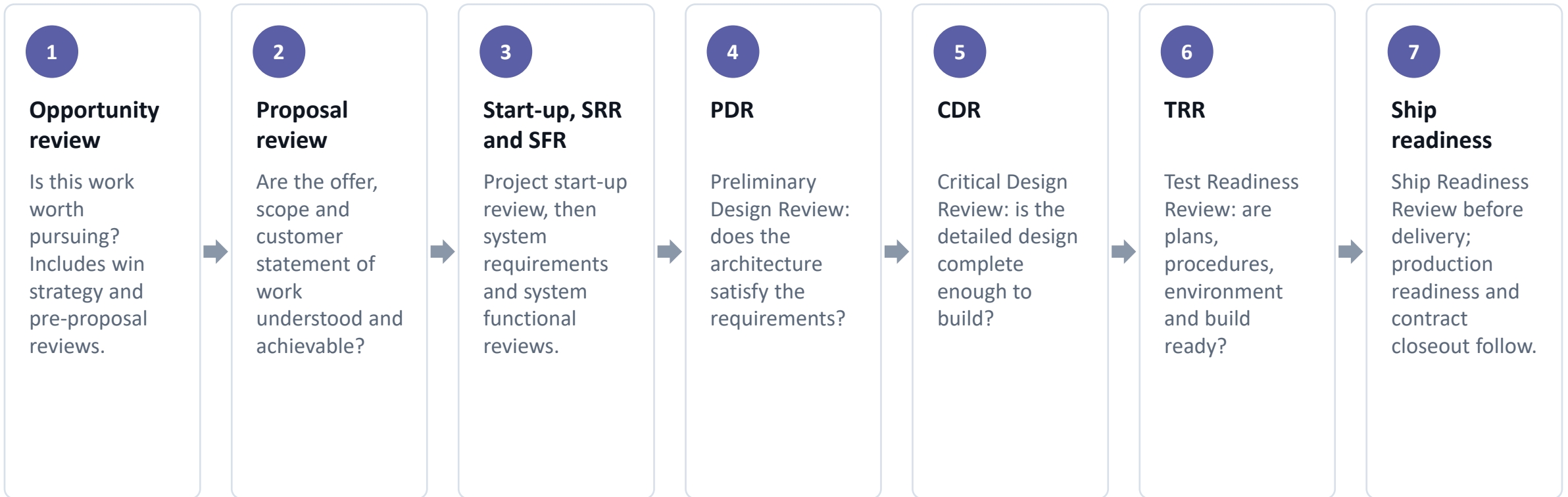
Defence-style document names from Tseytlin's lifecycle, mapped to the nearest KRISA D-template. The mapping is approximate, not official

Deliverable	Meaning	Phase	Nearest KRISA template
SRS	Software Requirements Specification	Software requirements	D03 SRS, with business needs in D02 BRS
IRS	Interface Requirements Specification	Software requirements	Interface sections of D03 SRS; D07 Pelan Integrasi Sistem
SDD	Software Design Description	Preliminary and detailed design	D04 SDS
ICD, IDD	Interface Control Document; Interface Design Description	Preliminary design	D08 Spesifikasi Integrasi Data
Test plans and procedures	Plans and step-by-step procedures for PQT, FAT and SAT	Design, before testing	D12 Pelan Induk Pengujian; D13 Pelan UAT/PAT
Code, unit test plans and results	Source code with unit test plans, procedures and results	Code and unit test	D10 Dokumentasi Kod Sumber
VDD	Version Description Document: exactly what a release contains	Integration and release	D18 Laporan Serahan Sistem (closest match)
PQT report	Results of qualification testing on the integrated software	Component and system test	D11 Laporan Ujian Sistem
FAT and SAT reports	Factory Acceptance Test and Site Acceptance Test results	Acceptance	D14 Laporan UAT/PAT; D16 Laporan Penamatan Ujian
ECP	Engineering Change Proposal leading to updates	Maintenance and support	No D-template; PPrISA10 Borang Pindaan

Source: Tseytlin, Raytheon SQA course (development cycle deliverables); KRISAv2 D01 to D18 template list

Review gates as quality gates

Tseytlin's Integrated Product Development System places a formal review at every major transition



Each gate needs entry and exit criteria agreed in the quality plan. Passing a gate should create a baseline, not just a meeting record.

Review checklist: systems engineering

- ☐ Are major functions defined in a bounded and unambiguous way?
- ☐ Are interfaces between system elements defined?
- ☐ Are performance bounds set for the system as a whole and for each element?
- ☐ Are design constraints established for each element?
- ☐ Has the best alternative been selected, and is the reasoning recorded?
- ☐ Is the solution technologically feasible?
- ☐ Is there an established mechanism to verify and validate the system?
- ☐ Are all system elements consistent with each other?

ShopFast: is the payment gateway boundary defined, and is the 2 second response target split across web, API and payment elements?

Review checklist: software project planning

- ☐ Is the software scope unambiguously defined and bounded?
- ☐ Is the terminology clear?
- ☐ Are resources adequate for the scope, and readily available?
- ☐ Are tasks properly defined and sequenced?
- ☐ Is the basis for cost estimation reasonable, and was it built from two different sources?
- ☐ Have historical productivity and quality data been used?
- ☐ Have differences between estimates been reconciled?
- ☐ Are the pre-established budgets and deadlines realistic?
- ☐ Is the schedule internally consistent?

ShopFast: does the plan include time for SRS review, test design and a UAT retest round, or only for building features?

Review checklist: software requirements analysis

- ☐ Is the information domain analysis complete, consistent and accurate?
- ☐ Is the required performance achievable within constraints from other system elements?
- ☐ Is problem partitioning complete?
- ☐ Are requirements consistent with schedule, resources and budget?
- ☐ Are external and internal interfaces properly defined?
- ☐ Are the validation criteria complete?
- ☐ Are all requirements traceable to the system level?
- ☐ Workshop addition: does every requirement have measurable acceptance criteria?
- ☐ Has prototyping been conducted with the customer?

Use this list in the ShopFast SRS review lab. Every question answered 'no' becomes an entry on the review issues list.

Baseline acceptance criteria

Set closure criteria in advance, so a baseline or test gate is accepted on evidence, not on pressure

- ☐ Criteria are written and agreed before the review or test starts, not negotiated at the meeting.
- ☐ Design baseline: PDR and CDR actions closed and interfaces approved.
- ☐ Source example: no priority 1 software trouble reports open at the FAT test readiness review.
- ☐ Test baseline: procedures dry-run by engineering and by SQA before the run for record.
- ☐ Requirements baseline: all review issues closed, or formally deferred with an owner and a date.
- ☐ Build frozen: no code changes are made during formal tests.
- ☐ Every requirement is uniquely identified and traceable to a business need and to a test.
- ☐ Approved baseline placed under configuration control; later changes go through change control.

ShopFast UAT entry example: zero open severity 1 defects, and every REQ-CHK requirement covered by at least one passed system test.

Practice: build quality before code

1

Fill a **QA Plan** template for ShopFast: scope, entry & exit criteria, Quality Gate.

2

Review a **deliberately flawed SRS**; flag ambiguity & untestable requirements.

3

Rewrite **5 requirements** into testable form + Acceptance Criteria (Given/When/Then).

4

Start the **RTM v0**, link each requirement to an ID.

TOOLS

[QA Plan template](#)[Sample SRS](#)[Review checklist](#)[RTM \(Sheet\)](#)[Jira](#) / Confluence

What you carry out



OUTPUT to

QA Plan + Requirement Review Report + RTM v0, carried into every later module.



KRISA 2.0-ALIGNED

BRS ([D2.1](#)) · **SRS**

Deliverables map to [KRISA 2.0 Ch.2](#) (Initiation) & [Ch.3](#) (Analysis).



FEEDS INTO

Module 3 onward

The RTM v0 becomes the backbone of later test design, automation & audit.

Note: KRISA 2.0 is Beta 2026 (JDN); deliverable numbering such as [D2.1](#) is still being refined. Confirm the version with your agency.

What the deliverables look like



QA Plan, outline

- ✓ Scope & objectives
- ✓ Test approach
- ✓ Entry criteria
- ✓ Exit criteria
- ✓ Quality Gate
- ✓ Roles & responsibilities
- ✓ Risks & assumptions



RTM, columns

- A Req ID
- B Requirement
- C Acceptance criteria
- D Test case ID
- E Status
- F Defect ID



Rewrite, sample

BEFORE

“Checkout must be fast and user-friendly.”

AFTER

“Given a cart with ≥ 1 item, when the user taps Pay, the confirmation shows within 3 seconds.”

◇ Illustrative samples for facilitation, adapt IDs, thresholds and columns to your agency’s templates.

Where these practices come from



KRISA 2.0 (Beta 2026), JDN

[Ch.2](#) Initiation to BRS ([D2.1](#)); [Ch.3](#) Analysis to SRS. Public-Sector SQA Portal, JDN.



ISO/IEC 25010 · 29119 · ISO 9001

Product quality model · software testing · quality management system.



CMMI

Process maturity model referenced in the audit module.



INVEST (Bill Wake)

Criteria for healthy, testable user stories.



Given / When / Then

Gherkin / BDD structure for acceptance criteria.



Cost-of-quality escalation

Widely-cited idea (Barry Boehm; IBM SSI), figures illustrative.

◇ KRISA 2.0 is a Beta; numbering (e.g. [D2.1](#)) is still being refined and KRISA 1.0 remains the baseline until finalised, confirm with your agency.

Abbreviations used in this module (1 of 7)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
AI	Artificial Intelligence	Software that can answer questions and write text or code by learning patterns from large amounts of data.
ANSI	American National Standards Institute	The body that approves official standards in the United States.
API	Application Programming Interface	A doorway that lets one program ask another program for data or an action, without using a screen.
ASVS	Application Security Verification Standard	An OWASP checklist of security controls a web application should have, grouped by level.
BDD	Behaviour-Driven Development	Describing the expected behaviour as simple Given, When, Then examples before the feature is built.
BR	Business Rule	A rule set by the business, for example who may view an order. Used in IDs such as BR-07.
BRS	Business Requirements Specification	The document that states what the business needs, written in business language. KRISA deliverable D02.
CDR	Critical Design Review	A formal check that the detailed design is complete enough to start building.
CMMI	Capability Maturity Model Integration	The newer model that replaced CMM. Used to improve and appraise how an organisation works.

Abbreviations used in this module (2 of 7)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
COTS	Commercial Off-The-Shelf	Ready-made software bought from a vendor instead of built in-house.
CSV	Comma-Separated Values	A simple table file that opens in Excel, with commas between the columns.
ECP	Engineering Change Proposal	A formal proposal to change an agreed baseline, usually with its cost and schedule impact.
FAT	Factory Acceptance Test	The formal test with the customer before the system is installed at the live site.
FR	Functional Requirement	Something the system must do. Used in IDs such as FR-03.
IBM	International Business Machines	A large technology company. Its research is often quoted on the cost of fixing defects late.
ICD	Interface Control Document	Describes exactly how two systems connect and exchange data.
ID	Identifier	A unique label, such as REQ-CHK-01, so that each item can be traced.
IDD	Interface Design Description	The detailed design of an interface between systems.

Abbreviations used in this module (3 of 7)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
IDOR	Insecure Direct Object Reference	A security hole where changing an ID in a request lets a user see someone else's data, such as another customer's order.
IEC	International Electrotechnical Commission	An international standards body that publishes many software standards together with ISO.
IEEE	Institute of Electrical and Electronics Engineers	A professional body that publishes software engineering standards such as IEEE 730 and IEEE 829.
INVEST	Independent, Negotiable, Valuable, Estimable, Small, Testable	A six-point checklist for writing good user stories and requirements.
IRS	Interface Requirements Specification	States what an interface between systems must do.
ISO	International Organization for Standardization	Publishes international standards such as ISO 9001 and ISO/IEC 25010.
JDN	Jabatan Digital Negara	The Malaysian government department that publishes KRISA and PPrISA.
KRISA	Kejuruteraan Sistem Aplikasi Sektor Awam	The Malaysian public sector guide for engineering application systems. KRISAv2 is the 2026 beta edition.
MIL	Military (as in MIL standards)	Standards written for United States defence projects, such as MIL-STD-498, which shaped many software documents.

Abbreviations used in this module (4 of 7)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
MIT	Massachusetts Institute of Technology	A university in the United States. Source of the Leveson course notes.
OWASP	Open Worldwide Application Security Project	A community that publishes free web security guides such as the OWASP Top 10.
PAT	Provisional Acceptance Test	The KRISA acceptance test done before provisional acceptance of the system, including performance checks.
PDF	Portable Document Format	A document file that looks the same on every computer.
PDPA	Personal Data Protection Act 2010	The Malaysian law on how personal data must be collected, used and protected.
PDR	Preliminary Design Review	An early formal check that the architecture can meet the requirements.
PPrISA	Panduan Pengurusan Projek ICT Sektor Awam	The Malaysian public sector guide for managing ICT projects. Codes such as PPrISA05 are its templates.
PPS	Pelan Pembangunan Sistem	The System Development Plan. KRISA deliverable D01.
PQT	Preliminary Qualification Test	A formal test showing that each software part meets its requirements before full system testing.

Abbreviations used in this module (5 of 7)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
PS	PowerShell prompt	When a slide shows PS>, type that command in Windows PowerShell.
QA	Quality Assurance	Preventing defects by setting up good processes and checking that people follow them.
QC	Quality Control	Finding defects in the product itself through testing and inspection.
QMS	Quality Management System	An organisation's policies, processes and records for quality, as described in ISO 9001.
REQ	Requirement	Prefix in requirement IDs such as REQ-CHK-01. CHK stands for checkout.
RM	Ringgit Malaysia	Malaysian currency.
RTM	Requirements Traceability Matrix	A table linking each requirement to its design, tests and defects, so nothing is missed.
SAT	Site Acceptance Test	The formal acceptance test carried out at the live site.

Abbreviations used in this module (6 of 7)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
SCM	Software Configuration Management	Tracking every version and change so the right items are built and released.
SDD	Software Design Description	The document that describes how the software is designed.
SDP	Software Development Plan	The plan that sets out how the software will be built: phases, methods, schedule and people.
SDS	Spesifikasi Reka Bentuk Sistem	The System Design Specification. KRISA deliverable D04.
SFR	System Functional Review	A formal check that the system functions are defined and agreed.
SQA	Software Quality Assurance	All the planned work that gives confidence the software meets its quality requirements.
SQC	Software Quality Control	Checks on the product itself: reviews, tests and audits.
SQP	Software Quality Plan	The plan stating which standards, reviews, tests and audits a project will use.

Abbreviations used in this module (7 of 7)

Every short form on these slides, explained in plain words. Keep this page handy during the lab.

Abbreviation	Stands for	What it means in plain words
SRR	System Requirements Review	A formal check that the requirements are complete, clear and agreed.
SRS	Spesifikasi Keperluan Sistem	The System (or Software) Requirements Specification: what the system must do. KRISA deliverable D03.
SSI	Systems Sciences Institute	An IBM research unit often quoted on the rising cost of late defect fixes.
TC	Test Case	Prefix in test case IDs such as TC-CHK-004.
TRR	Test Readiness Review	A check that the plans, procedures, environment and build are ready before formal testing starts.
UAT	User Acceptance Test	Users try the system to confirm it meets their needs before they accept it.
V&V	Verification and Validation	Verification asks: did we build it right? Validation asks: did we build the right thing?
VDD	Version Description Document	Lists exactly what is in a release: the version, the changes and the known issues.

References and templates for this module

Every line opens the official JDN document at the exact page. Templates download as Word or PDF.

WHERE IT IS IN THE GUIDES

- Bab 2, 2.4 Kajian Keperluan Bisnes (BRS)
page 10 · KRISAv2 Bab 2 Fasa Permulaan
- Bab 2, 2.11 Traceability Matrix (RTM) F1.8, Templat T2.8
page 56 · KRISAv2 Bab 2 Fasa Permulaan
- Bab 3, 3.8 Keperluan Bukan Fungsian F2.4
page 33 · KRISAv2 Bab 3 Fasa Analisis
- Bab 3, 3.10 Spesifikasi Keperluan Sistem (SRS)
page 42 · KRISAv2 Bab 3 Fasa Analisis
- Bab 8, 8.3.1 Gate Review dan Definition of Done
page 5 · KRISAv2 Bab 8 Penyelarasan DevOps
- PPrISA 2.0, 4.2.4 a) Pengurusan Kualiti
page 74 · PPrISA 2.0 Versi Beta Feb 2025

TEMPLATES TO USE

- D02
Spesifikasi Keperluan Bisnes (BRS)
WORD PDF
- D03
Spesifikasi Keperluan Sistem (SRS)
WORD PDF
- D12
Pelan Induk Pengujian
WORD PDF
- PPrISA05
Pelan Pengurusan Kualiti
WORD PDF
- PPrISA07
Pelan Pengurusan Risiko
WORD PDF



WRAP-UP · MODULES 1 & 2

What you leave with

- ✓ A shared quality vocabulary, QA/QC, V&V, Shift-Left.
- ✓ A QA Plan with entry/exit criteria and a Quality Gate.
- ✓ A Requirement Review Report on a flawed SRS.
- ✓ An RTM v0 linking requirements to tests to defects.

Every module runs on one shared project, ShopFast checkout, so you leave with a connected quality trail.

AA

Al-Abrar bin Abdul Wahid

SQA Trainer · KRISA 2.0 (JDN)

✉ abrar.abdulwahid@gmail.com

📞 +60 13-283 5415

🌐 [al-abrar-abdul-wahid](#)

M02 · W02 · 25 MINUTES · Hari 1, 12:30 hingga 12:55

Hands-On SQA-AI Assistant Workshop

AI-assisted requirement review and RTM

Let the AI statically review the flawed ShopFast SRS against the BRS, rewrite weak requirements as Given/When/Then and draft the RTM as CSV.

TOOL

opencode 1.18 in the lab folder · model: Big Pickle via OpenCode Zen (free, no API key) · synthetic ShopFast data only

[Step-by-step guide \(web\)](#)[Workshop slides \(W02\)](#)[Setup guide W00](#)[opencode docs](#)

AI-assisted requirement review and RTM

STEPS

- 1 Run `/review-srs` on two checkout requirements
- 2 Compare the AI findings with your team review
- 3 Run `/rewrite-gwt` and approve the file
- 4 Run `/rtm` and open the CSV

OUTPUT TO SAVE

`outputs/m02-gwt-rewrite.md`, `outputs/m02-rtm.csv`

TYPE THIS (`opencode>` in `opencode`, `PS>` in PowerShell)

```
opencode> /review-srs REQ-CHK-03 dan REQ-CHK-04  
opencode> /rewrite-gwt REQ-CHK-03 REQ-CHK-04  
opencode> /rtm REQ-CHK-01 hingga REQ-CHK-06
```

HUMAN CHECK BEFORE YOU ACCEPT IT

- Every finding quotes text that really exists in the SRS
- Each Given/When/Then has one observable Then
- RTM rows match the requirement IDs in the SRS, none invented

[Open step-by-step guide](#)

[Module hub](#)